



Zewail City for Science and Technology  
University of Science and Technology  
**Aerospace Engineering Program**

**“NACA 23022 Airfoil Project ”**

**Prepared By:**

Rehand Abdelhady Yassin      202000770

**Supervised By:**

Dr. Mohamed Madbouli  
Eng. Asmaa AlaaElDeen

**2024/2025**

## Table of Contents:

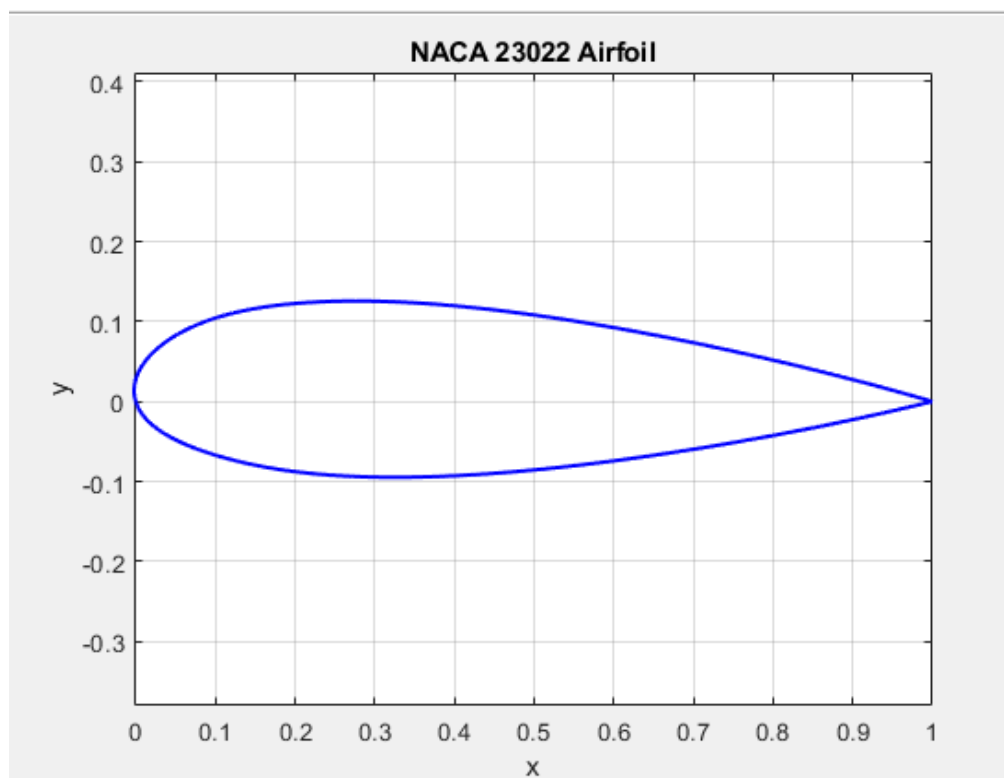
|                                                                                                                                 |           |
|---------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>I. Problem Definition.....</b>                                                                                               | <b>3</b>  |
| I.I. MATLAB.....                                                                                                                | 3         |
| I.II. Ansys.....                                                                                                                | 4         |
| I.III. Q-Blade.....                                                                                                             | 4         |
| <b>II. Suitable grid over the airfoil (H-grid).....</b>                                                                         | <b>4</b>  |
| II.I. Cord Length Definition Methodology.....                                                                                   | 5         |
| II.II. Grid Point Distribution in x-direction.....                                                                              | 5         |
| II.III. Airfoil Surface Definition.....                                                                                         | 5         |
| II.IV. Interpolation Between Grid Lines.....                                                                                    | 5         |
| II.V. Result.....                                                                                                               | 6         |
| II.V.1. MATLAB.....                                                                                                             | 6         |
| II.V.2. Ansys.....                                                                                                              | 6         |
| <b>III. Governing equation in the proposed body fitted grid.....</b>                                                            | <b>7</b>  |
| III.I. Define the coordinate transformation.....                                                                                | 7         |
| III.II. Apply the Chain Rule.....                                                                                               | 7         |
| III.III. Calculate the Metric Coefficients.....                                                                                 | 8         |
| III.IV. Transform Laplace's Equation.....                                                                                       | 8         |
| III.V. Discretize the Equation.....                                                                                             | 8         |
| <b>IV. Convergence history using LSOR showing “RMS” error with iteration number.....</b>                                        | <b>9</b>  |
| IV.I. Numerical Method LSOR.....                                                                                                | 9         |
| IV.II. Angle of Attack =5.....                                                                                                  | 10        |
| IV.III. Iterative Solution Process.....                                                                                         | 10        |
| IV.IV. Convergence Log.....                                                                                                     | 10        |
| IV.V. Results.....                                                                                                              | 10        |
| IV.V.1. Using MATLAB.....                                                                                                       | 10        |
| IV.V.2. Using Ansys.....                                                                                                        | 11        |
| <b>V. Velocity and pressure distributions on the upper and lower surfaces of the airfoil.....</b>                               | <b>12</b> |
| V.I. ISO Velocity Lines using Ansys.....                                                                                        | 12        |
| V.II. ISO Pressure Line using ansys.....                                                                                        | 13        |
| V.III. Velocity Vector using ansys.....                                                                                         | 13        |
| V.IV. Pressure Vector.....                                                                                                      | 14        |
| <b>VI. Determine the lift, drag, and moment coefficients <math>C_l</math> , <math>C_d</math> , <math>C_m(0.25c)</math>.....</b> | <b>17</b> |
| <b>VII. MATLAB Code.....</b>                                                                                                    | <b>20</b> |
| <b>VIII. References.....</b>                                                                                                    | <b>36</b> |

## I. Problem Definition

The objective is to solve the two-dimensional potential incompressible flow over a NACA 22023 airfoil using Laplace's equation for the stream function ( $\psi$ ). This involves constructing an H-grid, formulating the governing equation, and using the LSOR method for solution with a  $5^\circ$  angle of attack. The process includes iterating to convergence from an initial stream function value, producing convergence logs, iso-velocity and iso-pressure lines, and comparing velocity and pressure distributions with experimental data. The final results will include the lift, drag, and moment coefficients.

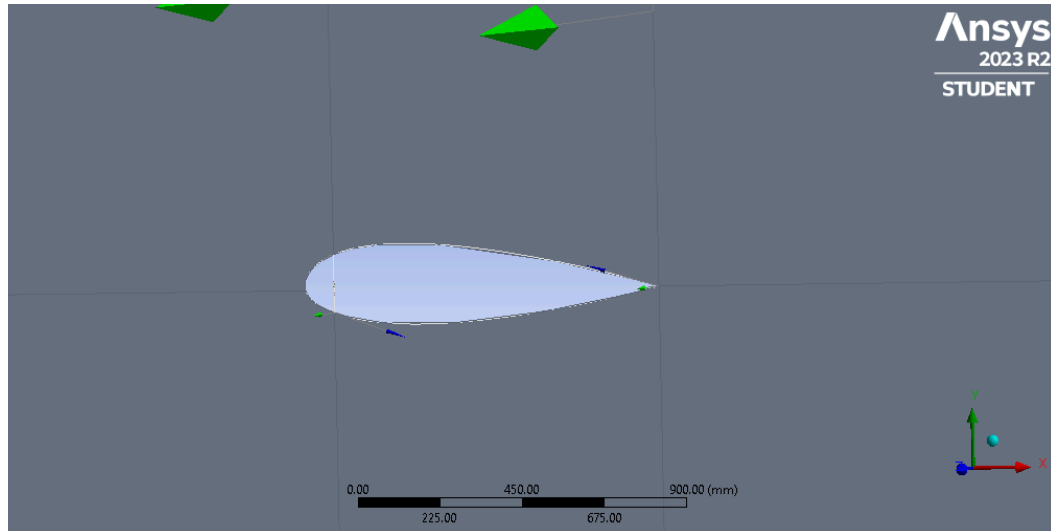
For [NACA 23022 Data](#)

### I.I. MATLAB



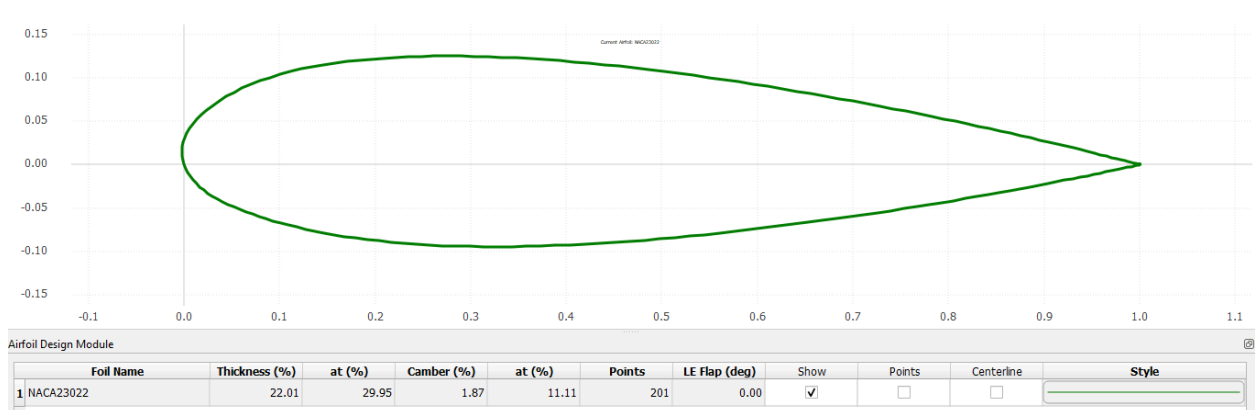
Fig(1)

## I.II. Ansys



Fig(2)

## I.III. Q-Blade



Fig(3)

## II. Suitable grid over the airfoil (H-grid)

The H-grid is constructed by defining a set of grid points in both the x and y directions. The grid points are distributed to ensure a smooth transition from the far-field boundary to the airfoil surface. The governing equations and methods used in the H-grid generation are outlined below:

## II.I. Cord Length Definition Methodology

The chord length,  $c$ , is set to 1 for scaling purposes.

## II.II. Grid Point Distribution in x-direction

The x-coordinates of the grid points are distributed linearly from the leading edge to the trailing edge of the airfoil.

$$x(ii, jj) = \left(\frac{c}{iimax-1}\right) * (ii - 1) - \frac{c}{2}$$

This ensures the grid is centered around the airfoil.

## II.III. Airfoil Surface Definition

The upper and lower surfaces of the airfoil are defined using the NACA 4-series equations for the camber line and thickness distribution. The equation for the thickness distribution  $y_t$ , is given by:

$$y_t = 5 \cdot toc \cdot (0.2969 \sqrt{x} - 0.126x - 0.3537x^2 + 0.2843x^3 - 0.1015x^4)$$

For the upper surface:

$$y_{upper}(ii, jjair) = y_t$$

For the lower surface :

$$y_{lower}(ii, jjair) = - y_t$$

## II.IV. Interpolation Between Grid Lines

The y-coordinates between the airfoil surface and the far-field boundary are interpolated linearly.

$$y(ii, jj) = y(ii, 1) + \frac{jj-1}{jjair-1} \cdot (y(ii, jjair) - y(ii, 1))$$

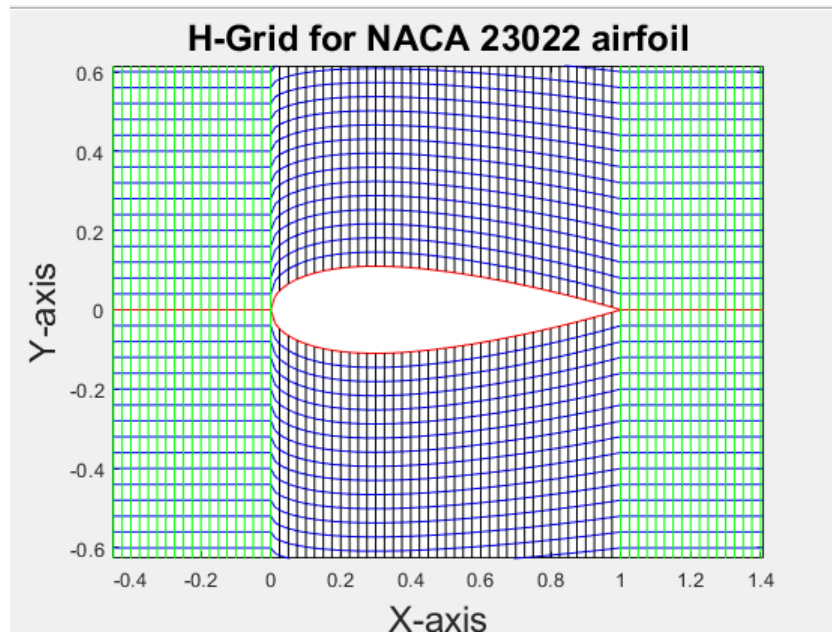
for  $jj < jjair$  and  $jj < jjair$ , and

$$y(ii, jj) = y(ii, jjair) + \frac{jj-jjair}{jjmax-jjair} \cdot (y(ii, jjmax) - y(ii, jjair))$$

for  $jj > jjair$  and  $jj > jjair$ .

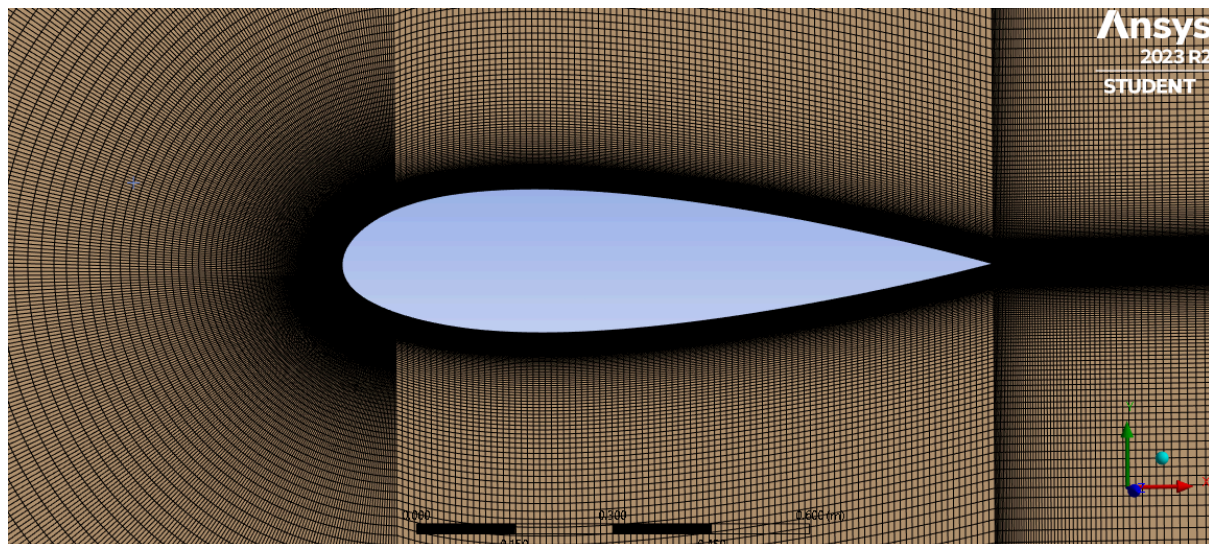
## II.V. Result

### II.V.1. MATLAB



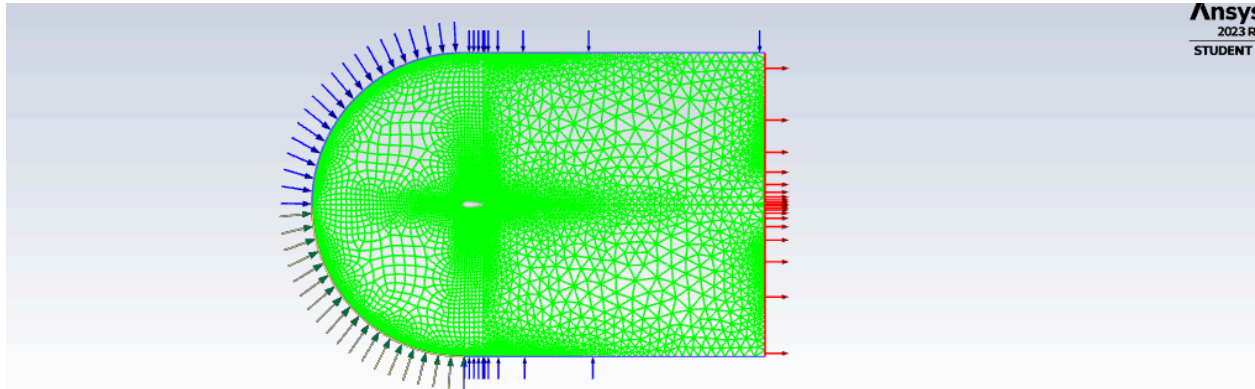
Fig(4)

### II.V.2. Ansys



Fig(5)

### Meshing Using Ansys



Fig(6)

## III. Governing equation in the proposed body fitted grid

### III.I. Define the coordinate transformation

- The body-fitted coordinates( $\eta_1, \eta_2$ ) are defined such that they conform to the geometry of the airfoil.  $\eta_1$  might be along the airfoil surface and  $\eta_2$  perpendicular to it.
- Assume the transformation between Cartesian and body-fitted coordinates is given by  $x = x(\eta_1, \eta_2)$  and  $y = y(\eta_1, \eta_2)$

### III.II. Apply the Chain Rule

Laplace's equation in Cartesian coordinates is

$$\nabla^2 \psi = \frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = 0$$

Using the chain rule, express the partial derivatives in terms of the body-fitted coordinates ( $\eta_1, \eta_2$ )

$$\frac{\partial \psi}{\partial x} = \frac{\partial \psi}{\partial \eta_1} \frac{\partial \eta_1}{\partial x} + \frac{\partial \psi}{\partial \eta_2} \frac{\partial \eta_2}{\partial x}$$
$$\frac{\partial \psi}{\partial y} = \frac{\partial \psi}{\partial \eta_1} \frac{\partial \eta_1}{\partial y} + \frac{\partial \psi}{\partial \eta_2} \frac{\partial \eta_2}{\partial y}$$

### III.III. Calculate the Metric Coefficients

The metric coefficients describe the transformation between the coordinate systems.

$$\frac{\partial \eta_1}{\partial x}, \frac{\partial \eta_1}{\partial y}, \frac{\partial \eta_2}{\partial x}, \frac{\partial \eta_2}{\partial y}$$

### III.IV. Transform Laplace's Equation

Substitute the expressions from the chain rule into Laplace's equation to transform it into the body-fitted coordinates:

$$\nabla^2 \psi = \frac{\partial}{\partial \eta_1} \left( \frac{\partial \eta_1}{\partial x} \frac{\partial \psi}{\partial \eta_1} + \frac{\partial \eta_1}{\partial y} \frac{\partial \psi}{\partial \eta_2} \right) + \frac{\partial}{\partial \eta_2} \left( \frac{\partial \eta_2}{\partial x} \frac{\partial \psi}{\partial \eta_1} + \frac{\partial \eta_2}{\partial y} \frac{\partial \psi}{\partial \eta_2} \right) = 0$$

Simplifying the expression to get the transformed Laplace's equation

$$\nabla^2 \psi = \frac{\partial^2 \psi}{\partial \eta_1^2} + \frac{\partial^2 \psi}{\partial \eta_2^2} = 0$$

### III.V. Discretize the Equation

From Continuous Laplace's Equation and transformation to body fitted coordinates

$$\nabla^2 \psi = \frac{\partial^2 \psi}{\partial \eta_1^2} + \frac{\partial^2 \psi}{\partial \eta_2^2} = 0$$



Using finite difference approximation specifically centered difference formula

$$\frac{\partial^2 \psi}{\partial \eta_1^2} \approx \frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{\Delta \eta_1^2}$$

$$\frac{\partial^2 \psi}{\partial \eta_2^2} \approx \frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{\Delta \eta_2^2}$$

Combining these approximations, we get

$$\frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{\Delta \eta_1^2} + \frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{\Delta \eta_2^2} = 0$$

rearrange the above equation to solve for the stream function  $\psi$

$$\psi_{i,j} = \frac{\Delta \eta_2^2(\psi_{i+1,j} + \psi_{i-1,j}) + \Delta \eta_1^2(\psi_{i,j+1} + \psi_{i,j-1})}{2(\Delta \eta_1^2 + \Delta \eta_2^2)}$$

## IV. Convergence history using LSOR showing “RMS” error with iteration number

To solve the discretized governing equation, we choose the LSOR (Line Successive Over-Relaxation) method. The LSOR method is an iterative technique used to solve linear systems of equations, providing improved convergence rates over the standard Gauss-Seidel method by introducing a relaxation factor.

### IV.I. Numerical Method LSOR

- LSOR is used to accelerate the convergence of the iterative solution by applying a relaxation factor  $\omega$
- The standard Gauss-Seidel iteration is modified by the relaxation factor to update the solution:

$$\psi_{i,j}^{(new)} = (1 - \omega)\psi_{i,j}^{(old)} + \frac{\omega}{2(\Delta \eta_1^2 + \Delta \eta_2^2)} \left( \Delta \eta_2^2 (\psi_{i+1,j}^{(old)} + \psi_{i-1,j}^{(new)}) + \Delta \eta_1^2 (\psi_{i,j+1}^{(old)} + \psi_{i,j-1}^{(new)}) \right)$$

## IV.II. Angle of Attack =5

The boundary conditions are adjusted to account for an angle of attack of  $\alpha = 5^\circ$  ensuring that the flow approaches the airfoil at this angle. We initialize the stream function  $\psi$  to zero across all grid points

$$\psi_{i,j}^{(0)} = 0 \quad \text{for all } (i, j)$$

## IV.III. Iterative Solution Process

The LSOR method is iterated until the solution converges, with convergence determined by the change in  $\psi$  between iterations falling below a specified tolerance. The process continues until:

$$|\psi_{i,j}^{(\text{new})} - \psi_{i,j}^{(\text{old})}| < 10^{-6}$$

## IV.IV. Convergence Log

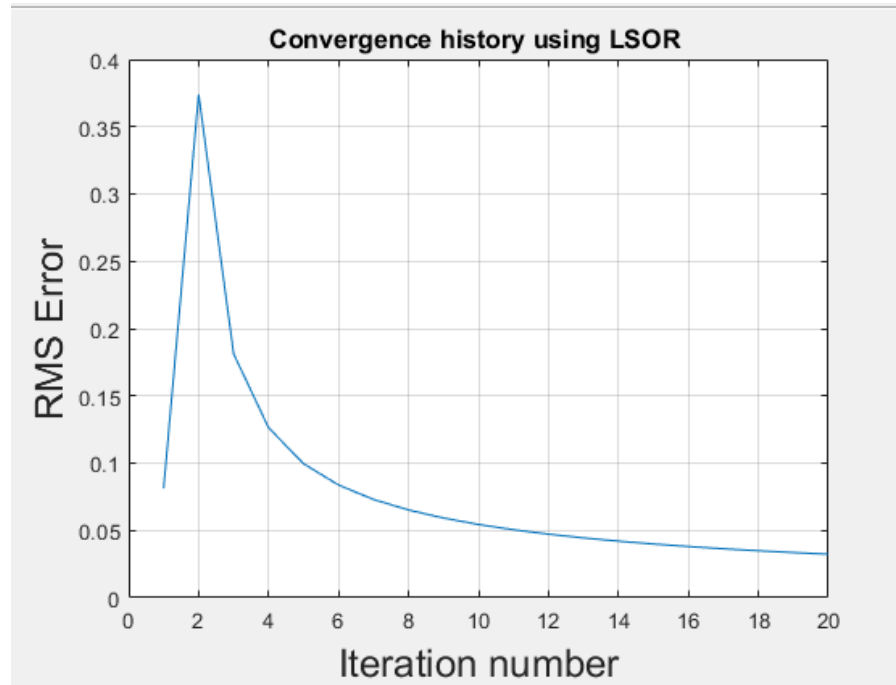
A convergence log is maintained by recording the RMS error at each iteration:

$$\text{RMS error} = \sqrt{\frac{1}{N} \sum_{i,j} \left( \psi_{i,j}^{(\text{new})} - \psi_{i,j}^{(\text{old})} \right)^2}$$

The RMS error is plotted against the iteration number to visualize and ensure convergence.

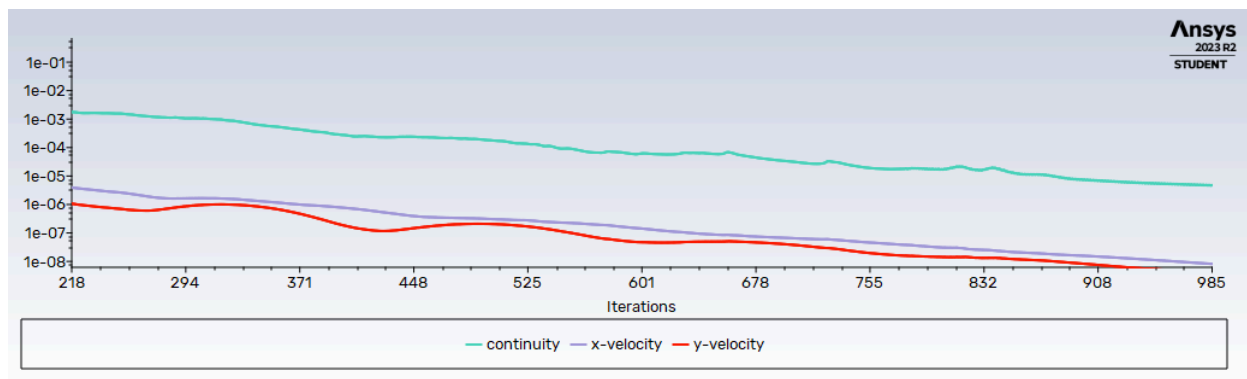
## IV.V. Results

### IV.V.1. Using MATLAB

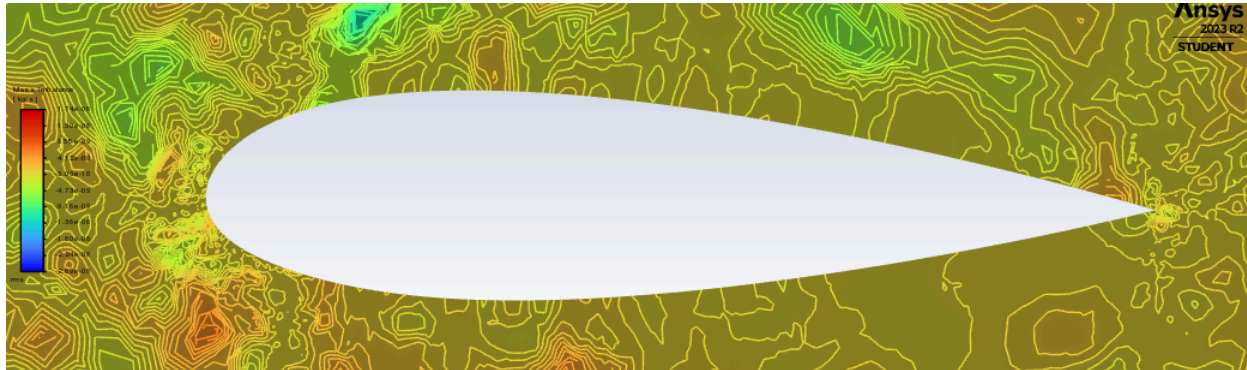


Fig(7)

#### IV.V.2. Using Ansys



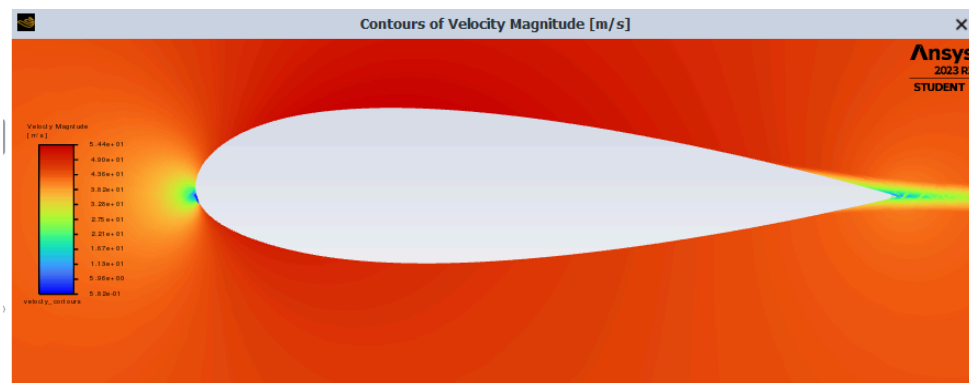
Fig(8)



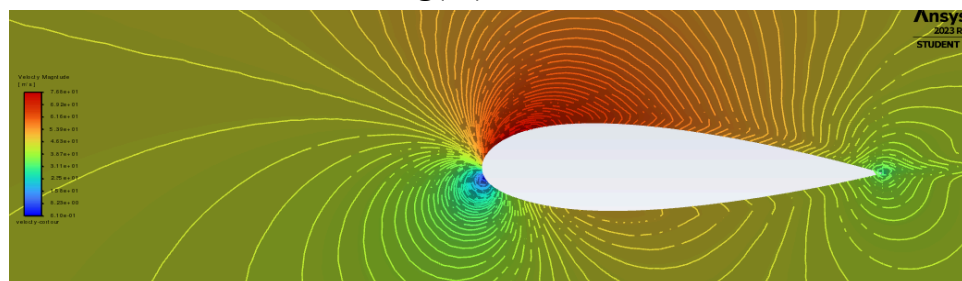
Fig(9): Residuals Countour on Ansys

## V. Velocity and pressure distributions on the upper and lower surfaces of the airfoil

### V.I. ISO Velocity Lines using Ansys

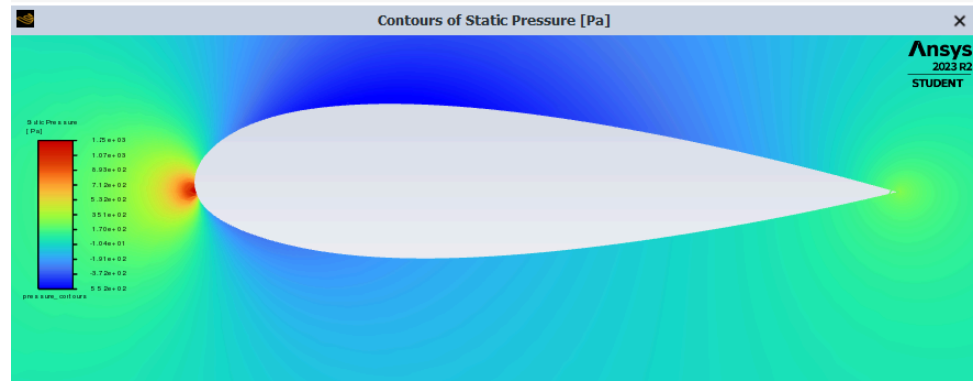


Fig(10)

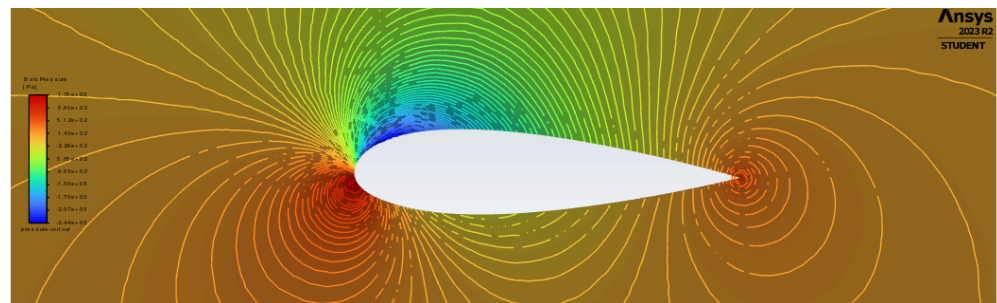


Fig(11)

## V.II. ISO Pressure Line using ansys

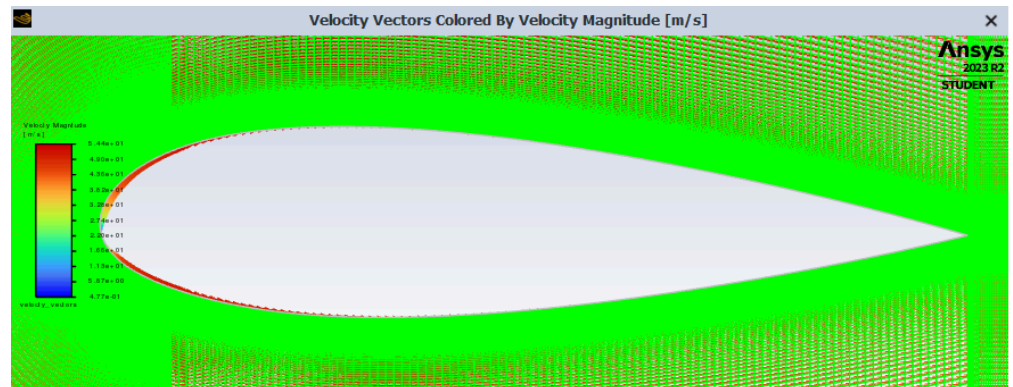


Fig(12)

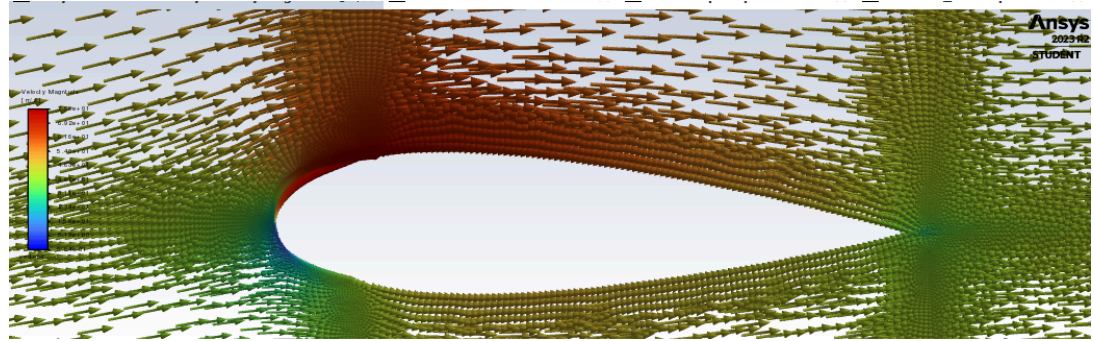


Fig(13)

## V.III. Velocity Vector using ansys



Fig(14)



Fig(15)

#### V.IV. Pressure Vector

The velocity components  $u$  and  $v$  can be derived from the stream function

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}$$

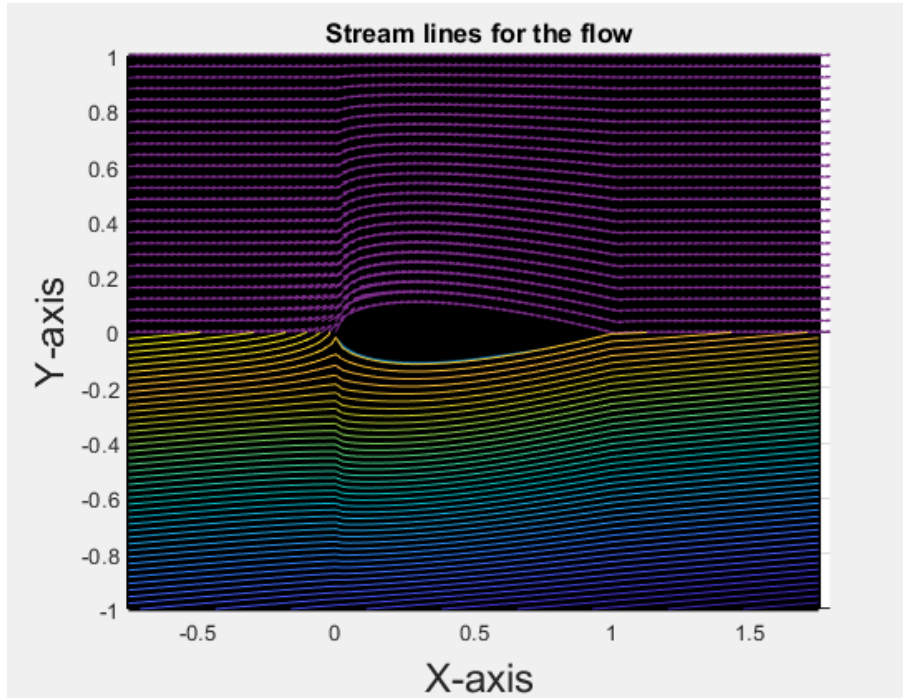
The velocity magnitude is given by:

$$V = \sqrt{u^2 + v^2}$$

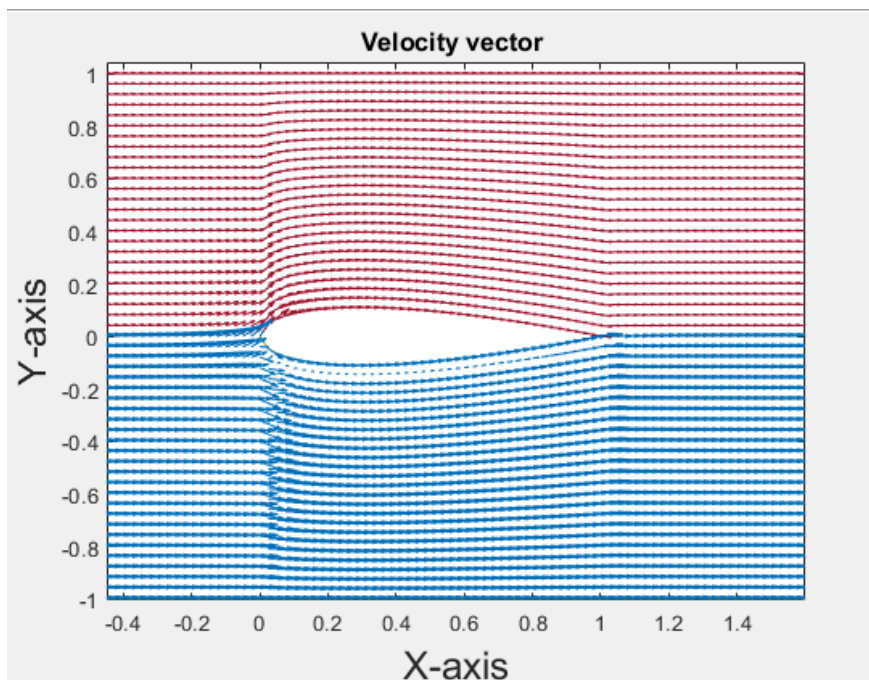
The pressure coefficient is calculated using the velocity magnitude

$$C_p = 1 - \left( \frac{V}{V_\infty} \right)^2$$

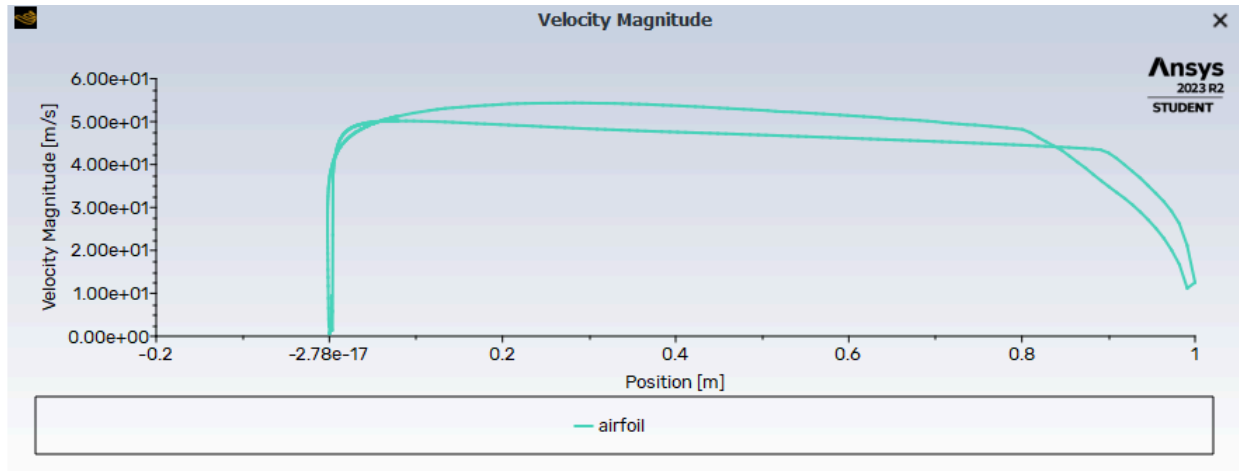




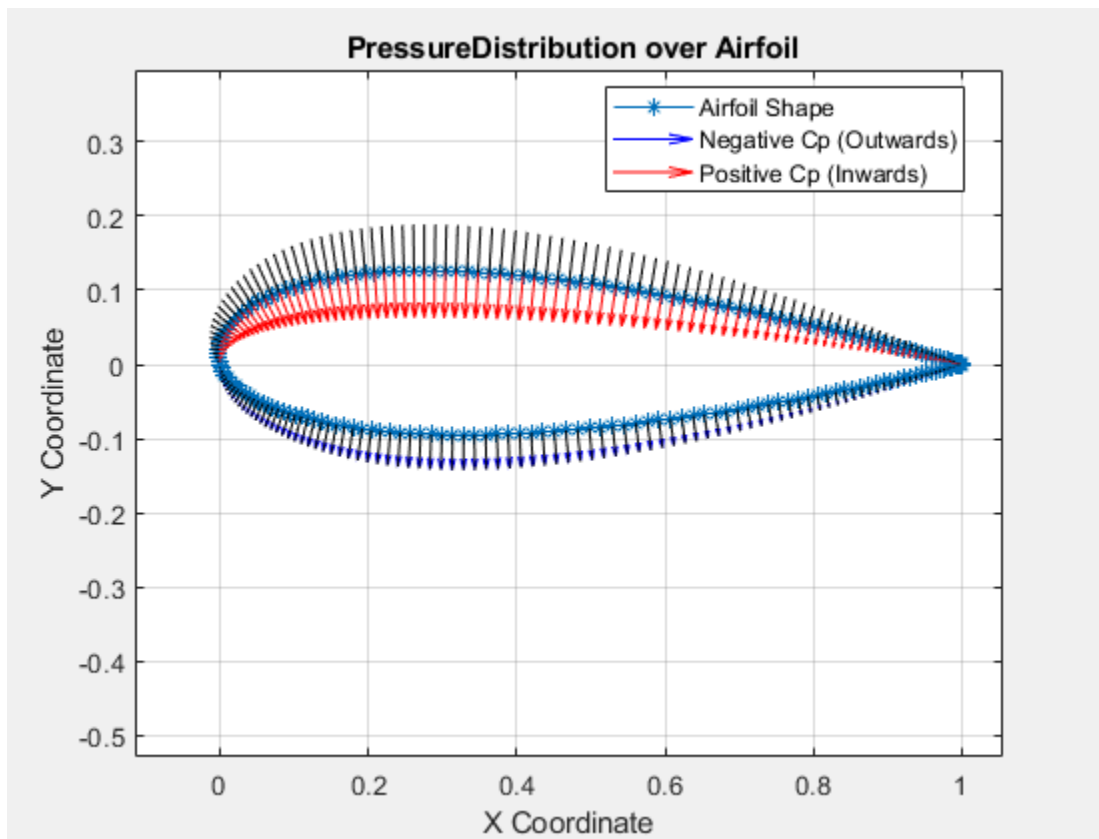
Fig(16)



Fig(17)

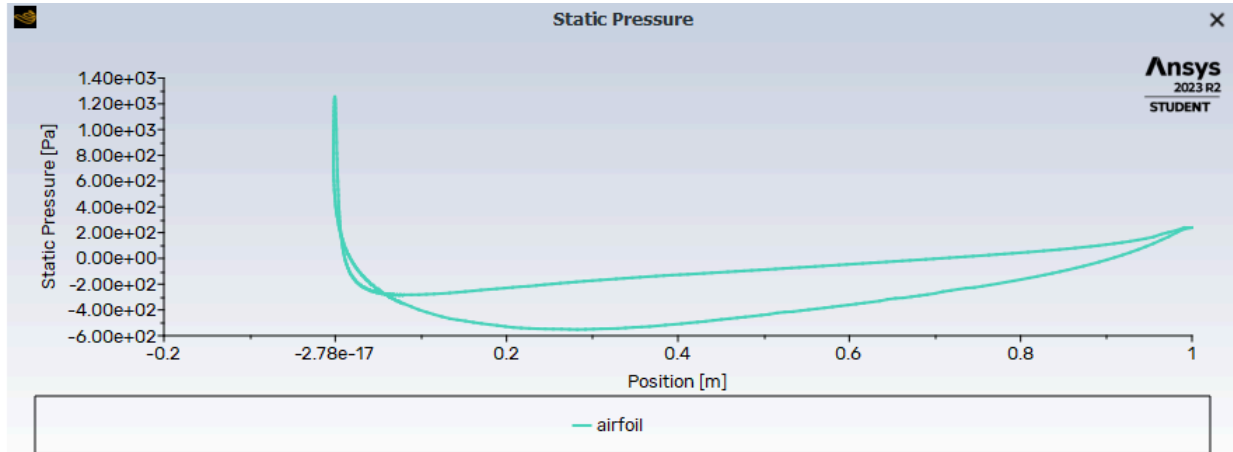


Fig(18)



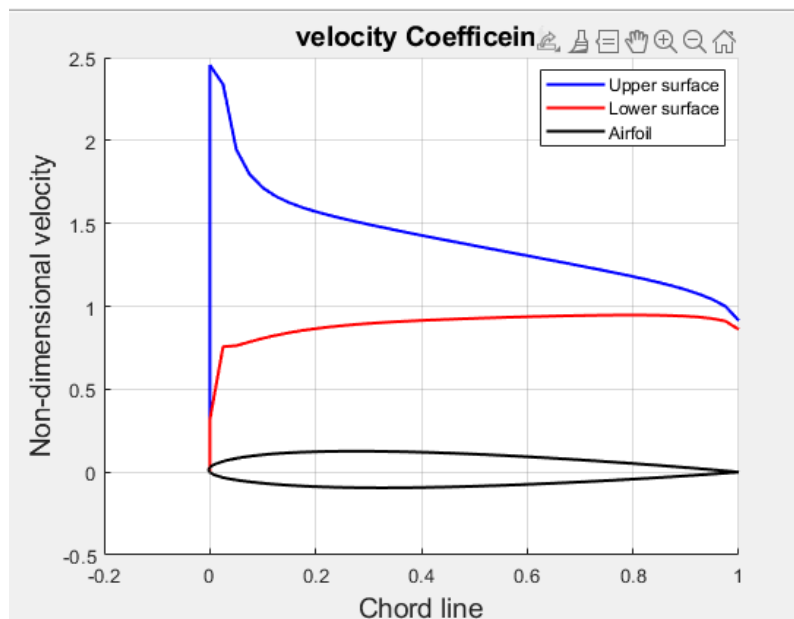
Fig(19)



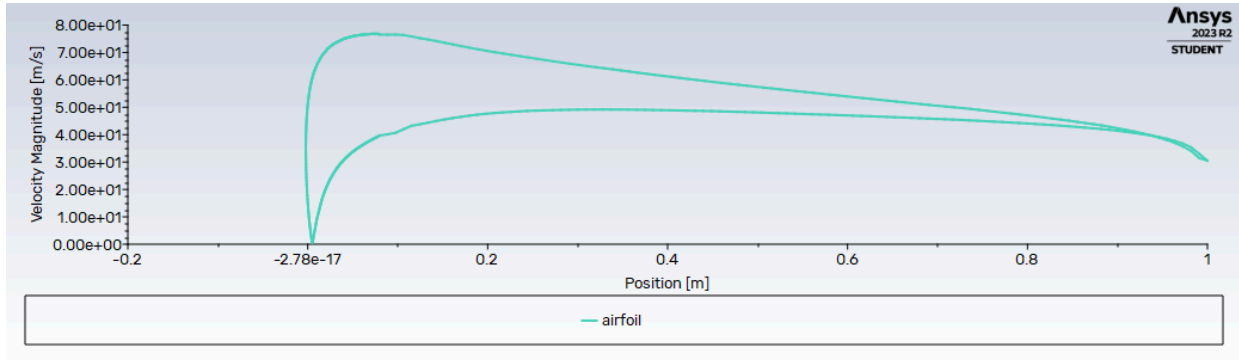


Fig(20)

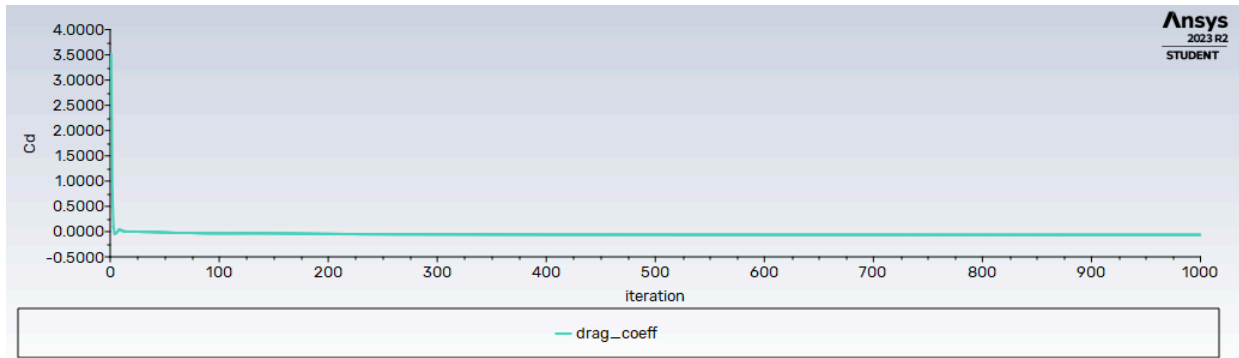
## VI. Determine the lift, drag, and moment coefficients $C_l$ , $C_d$ , $C_m(0.25c)$



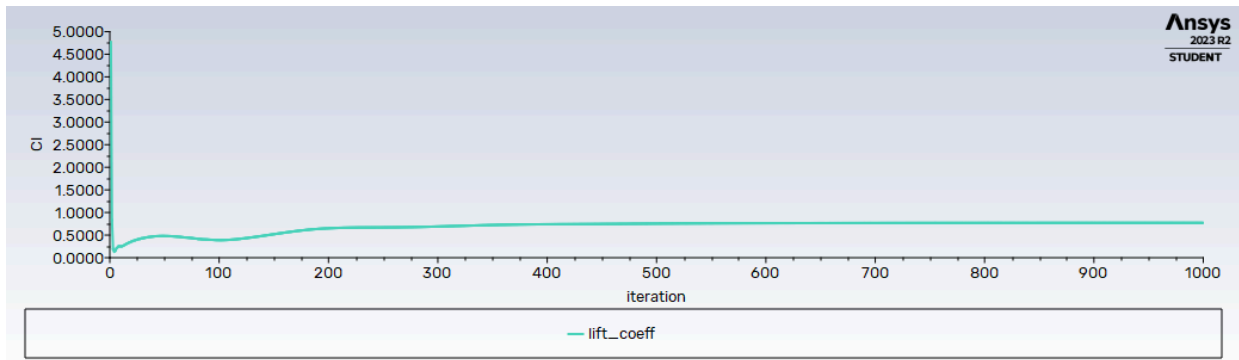
Fig(21)



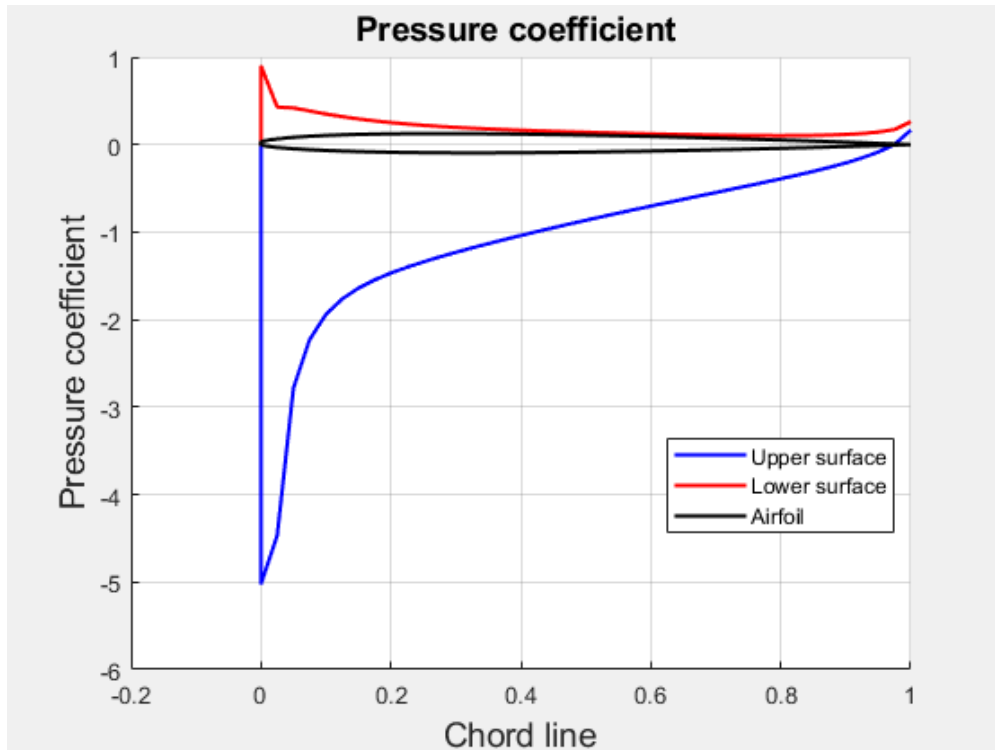
Fig(22)



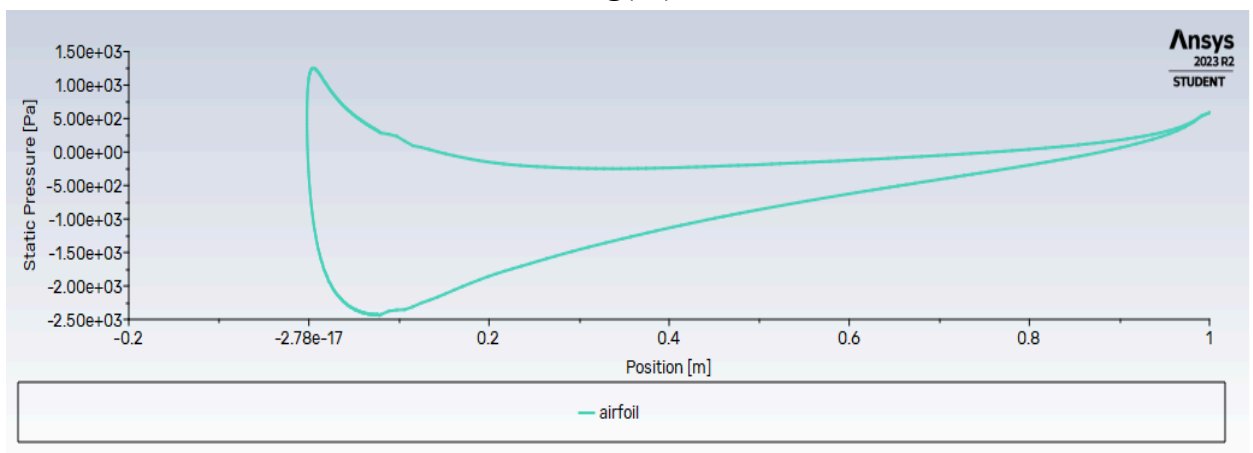
Fig(23)



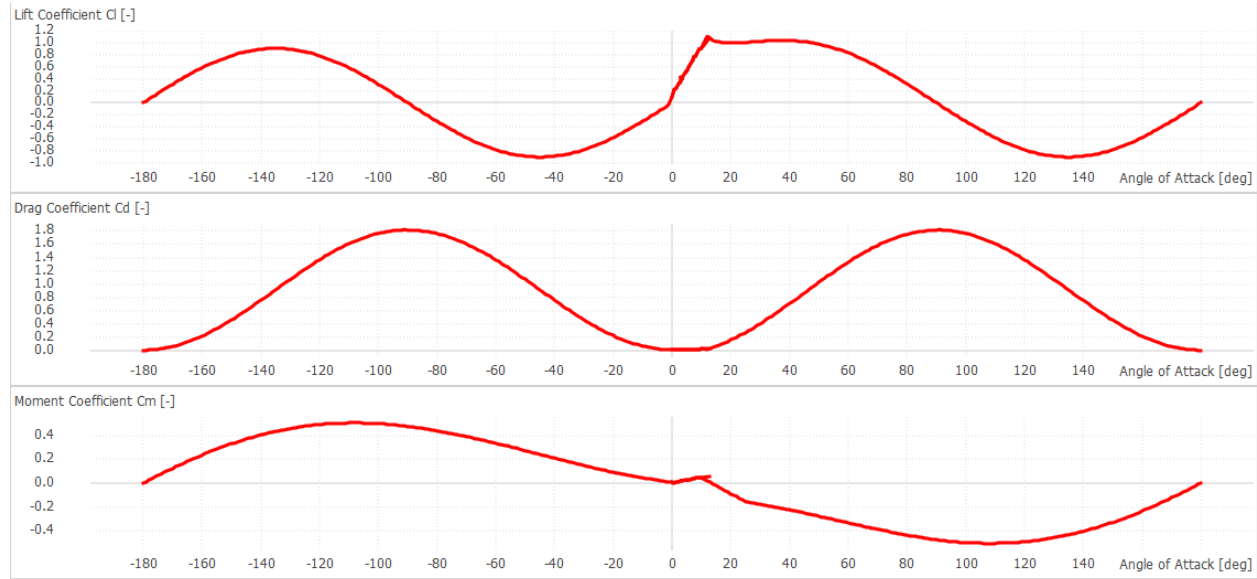
Fig(24)



Fig(25)



Fig(26)



Fig(27): Comparing Results using Q-Blade

Lift coefficient ( $c_l$ ): 1.278  
Drag coefficient ( $c_d$ ): -0.25541  
Moment coefficient ( $cm_{0.25c}$ ): 0.0049353

## VII. MATLAB Code

All [MATLAB Codes](#)

### [1]coef.m

```
function [c11, c12, c22] = coef(ip, jp)
    global x y d1 d2
    d1x = (x(ip + 1, jp) - x(ip - 1, jp)) / d1;
    d1y = (y(ip + 1, jp) - y(ip - 1, jp)) / d1;
    d2x = (x(ip, jp + 1) - x(ip, jp - 1)) / d2;
    d2y = (y(ip, jp + 1) - y(ip, jp - 1)) / d2;
    jaco = d1x * d2y - d1y * d2x;
    c11 = (d2x * d2x + d2y * d2y) / jaco;
    c12 = -(d1x * d2x + d1y * d2y) / jaco;
    c22 = (d1x * d1x + d1y * d1y) / jaco;
end
```

### [2] plot\_iso\_lines.m

```
function plot_iso_lines(x, y, cp)
    % Calculate gradients and normalize
```

```
dx = gradient(x);
dy = gradient(y);
ds = sqrt(dx.^2 + dy.^2);
dx = dx ./ ds;
dy = dy ./ ds;

% Scale the vectors using Cp to represent the Pressure Distribution
dx = abs(cp) .* dx;
dy = abs(cp) .* dy;

% Scaling factor for the length of Arrow Vector
scale = 0.5;

% Determine the point x1, y1 representing the other end of Arrow Vector
x1 = x + scale * dy;
y1 = y - scale * dx;

% Plot the Airfoil
figure;
plot(x, y, '*-');
hold on;

% Plot all negative cp value Arrow pointing outwards and vice versa
ind1 = cp < 0;

% Set a broader limit for axis, to avoid any warning/error by arrow command
axis([-5 5 -5 5]);

% Plot all negative cp using blue color
quiver(x(ind1), y(ind1), scale * dy(ind1), -scale * dx(ind1), 0, 'b');

% Plot all positive cp using red color
quiver(x(~ind1), y(~ind1), -scale * dy(~ind1), scale * dx(~ind1), 0, 'r');

% Make axis equal to visualize the Airfoil Shape properly
axis equal;

% Plot a Boundary to Beautify the Cp Vectors!!!
plot([x x1], [y y1], 'k', 'linewidth', 0.1);

% Add grid and axis
grid on;
box on;
```

```
% Add titles and labels
title(' PressureDistribution over Airfoil');
xlabel('X Coordinate');
ylabel('Y Coordinate');
legend({'Airfoil Shape', 'Negative Cp (Outwards)', 'Positive Cp (Inwards)'}, 'Location', 'Best');

% Save the figure
saveas(gcf, 'Cp_Distribution.png');
end

[3] geom.m
function geom
    global xairfoil yairfoil x y imax jmax jair il it cord yal yau

    % Read airfoil data from file
    data = readmatrix('airfoil_data.txt');
    xairfoil = data(:, 1);
    yairfoil = data(:, 2);

    % Scale the airfoil coordinates to match the chord length
    cord = 1;
    xairfoil = xairfoil * cord;
    yairfoil = yairfoil * cord;

    % Initialize x and y arrays
    x = zeros(2 * imax - 1, 2 * jmax - 1);
    y = zeros(2 * imax - 1, 2 * jmax - 1);
    % Determine the x(i,j) and y(i,j) for all points of H-grid
    iil = 2 * il - 1;
    iit = 2 * it - 1;
    iimax = 2 * imax - 1;
    jjmax = 2 * jmax - 1;
    jjair = 2 * jair - 1;
    for jj = 1:jjmax
        for ii = 1:iimax
            x(ii, jj) = (cord / (iit - iil)) * (ii - iil);
        end
    end
    end
    toc = 0.22;
    for ii = 1:iil
        y(ii, jjair) = 0;
    end
    for ii = iil:iit
```

```

    xp = x(ii, jjair);
    y(ii, jjair) = -5 * toc * (.2969 * sqrt(xp) - .126 * xp - .3537 * xp ^ 2 + .2843 * xp ^ 3 - .1015 * xp ^ 4);
end
for ii = iit:iimax
    y(ii, jjair) = 0;
end
for ii = 1:iimax
    yal(ii) = y(ii, jjair);
end
for ii = 1:iimax
    y(ii, 1) = -cord;
end
for ii = 1:iimax
    for jj = 2:jjair - 1
        y(ii, jj) = y(ii, 1) + (jj - 1) * (y(ii, jjair) - y(ii, 1)) / (jjair - 1);
    end
end
toc = 0.22;
for ii = 1:iil
    y(ii, jjair) = 0;
end
for ii = iil:iit
    xp = x(ii, jjair);
    y(ii, jjair) = 5 * toc * (.2969 * sqrt(xp) - .126 * xp - .3537 * xp ^ 2 + .2843 * xp ^ 3 - .1015 * xp ^ 4);
end
for ii = iit:iimax
    y(ii, jjair) = 0;
end
for ii = 1:iimax
    yau(ii) = y(ii, jjair);
end
for ii = 1:iimax
    y(ii, jjmax) = cord;
end
for ii = 1:iimax
    for jj = jjair + 1:jjmax - 1
        y(ii, jj) = y(ii, jjair) + (jj - jjair) * (y(ii, jjmax) - y(ii, jjair)) / (jjmax - jjair);
    end
end
% Plot the H-Grid
figure
for j = 1:jair - 1
    jj = 2 * j - 1;

```



```
x1 = x(:, jj);  
y1 = y(:, jj);  
plot(x1, y1, 'b'); % Change color to blue  
hold on;  
end  
jj = jjair;  
x1 = x(:, jj);  
y1 = yal(:);  
plot(x1, y1, 'r'); % Change color to red  
hold on;  
x1 = x(:, jj);  
y1 = yau(:);  
plot(x1, y1, 'r'); % Change color to red  
hold on;  
for j = jair + 1:jmax  
    jj = 2 * j - 1;  
    x1 = x(:, jj);  
    y1 = y(:, jj);  
    plot(x1, y1, 'b'); % Change color to blue  
    hold on;  
end  
y(:, jjair) = yal(:);  
for i = 1:il  
    ii = 2 * i - 1;  
    x1 = x(ii, :);  
    y1 = y(ii, :);  
    plot(x1, y1, 'g'); % Change color to green  
    hold on;  
end  
y(:, jjair) = yal(:);  
for i = il + 1:it - 1  
    ii = 2 * i - 1;  
    for j = 1:jair  
        jj = 2 * j - 1;  
        x2(j) = x(ii, jj);  
        y2(j) = y(ii, jj);  
    end  
    plot(x2, y2, 'k'); % Change color to black  
    hold on;  
end  
y(:, jjair) = yau(:);  
for i = il + 1:it - 1  
    ii = 2 * i - 1;
```

```

for j = jair:jmax
    jj = 2 * j - 1;
    k = j - jair + 1;
    x2(k) = x(ii, jj);
    y2(k) = y(ii, jj);
end
plot(x2, y2, 'k'); % Change color to black
hold on;
end
for i = it:imax
    ii = 2 * i - 1;
    x1 = x(ii, :);
    y1 = y(ii, :);
    plot(x1, y1, 'g'); % Change color to green
    hold on;
end
xlabel('X-axis', 'fontsize', 18)
ylabel('Y-axis', 'fontsize', 18)
title('H-Grid for NACA 23022 airfoil', 'fontsize', 18)
end
[4] L_SOR.m
function L_SOR
global y imax jmax jair il it yal yau ps psp r
iimax = 2 * imax - 1;
jjmax = 2 * jmax - 1;
jjair = 2 * jair - 1;
for j = 2:jmax-1
    if j == jair - 1; for ii = 1:iimax; y(ii, jjair) = yal(ii); end; end
    if j == jair + 1; for ii = 1:iimax; y(ii, jjair) = yau(ii); end; end
    b = zeros(imax, 1);
    d = zeros(imax, 1);
    a = zeros(imax, 1);
    c = zeros(imax, 1);
    b(1) = 0; d(1) = 1; a(1) = 0; c(1) = ps(1, j);
    b(imax) = 0; d(imax) = 1; a(imax) = 0; c(imax) = ps(imax, j);
    for i = 2:imax-1
        ii = 2 * i - 1;
        jj = 2 * j - 1;
        ip = ii + 1; jp = jj; [c11ip, c12ip, ~] = coef(ip, jp);
        ip = ii - 1; jp = jj; [c11im, c12im, ~] = coef(ip, jp);
        ip = ii; jp = jj + 1; [~, c12jp, c22jp] = coef(ip, jp);
        ip = ii; jp = jj - 1; [~, c12jm, c22jm] = coef(ip, jp);
        sij = c11ip + c11im + r * r * (c22jp + c22jm);
    end
end

```

```

sim = c11im - r * r * (c12jp - c12jm) / 4;
sip = c11ip + r * r * (c12jp - c12jm) / 4;
sjm = r * r * c22jm - r * (c12ip - c12im) / 4;
sjp = r * r * c22jp + r * (c12ip - c12im) / 4;
b(i) = -sim;
d(i) = sij;
a(i) = -sip;
c(i) = sjm * psp(i, j - 1) + sjp * ps(i, j + 1);
if (j == jair) && (i >= il) && (i <= it)
    b(i) = 0;
    d(i) = 1;
    a(i) = 0;
    c(i) = psp(i, j);
end
end
ps_p = tri_sol(a, b, c, d, imax);
for i = 1:imax
    psp(i, j) = ps_p(i);
end
end
ps_p = tri_sol(a, b, c, d, imax);
for i = 1:imax
    psp(i, j) = ps_p(i);
end
end
[5] PROJECT.m
% Read data from the file
data = readmatrix('airfoil_data.txt');
% Separate the data into x and y coordinates
x = data(:, 1);
y = data(:, 2);
% Plot the airfoil
figure;
plot(x, y, 'b-', 'LineWidth', 1.5);
axis equal;
grid off;
title('NACA 23022 Airfoil');
xlabel('x');
ylabel('y');
[6] results.m
function results()
    global x y imax jmax jair il it cord yal yau ps psp dx dy r d1 d2 omega Vinf cosa sina
    cord = 1.0; % Define chord length here

```

```
% Read airfoil data from file
data = readmatrix('airfoil_data.txt');
xairfoil = data(:, 1);
yairfoil = data(:, 2);

% Ensure airfoil coordinates fit within the domain
xairfoil = xairfoil * cord;
yairfoil = yairfoil * cord;
iimax = 2 * imax - 1;
jjmax = 2 * jmax - 1;
jjair = 2 * jair - 1;
uxinf = Vinf * cosa;
uyinf = Vinf * sina;
a_vx = zeros(imax, jmax);
a_vy = zeros(imax, jmax);
% Boundary conditions
i = imax;
for j = 1:jmax
    a_vx(i, j) = uxinf;
    a_vy(i, j) = uyinf;
end
i = 1;
for j = 1:jmax
    a_vx(i, j) = uxinf;
    a_vy(i, j) = uyinf;
end
j = 1;
for i = 1:imax
    a_vx(i, j) = uxinf;
    a_vy(i, j) = uyinf;
end
j = jmax;
for i = 1:imax
    a_vx(i, j) = uxinf;
    a_vy(i, j) = uyinf;
end
% Internal points
for i = 2:imax-1
    for j = 2:jmax-1
        ii = 2 * i - 1;
        jj = 2 * j - 1;
        d1x = (x(ii + 1, jj) - x(ii - 1, jj)) / d1;
        d1y = (y(ii + 1, jj) - y(ii - 1, jj)) / d1;
```

```

d2x = (x(ii, jj + 2) - x(ii, jj)) / d2;
d2y = (y(ii, jj + 2) - y(ii, jj)) / d2;
jaco = d1x * d2y - d1y * d2x;
et1x = d2y / jaco;
et1y = -d2x / jaco;
et2x = -d1y / jaco;
et2y = d1x / jaco;
d1u = (ps(i + 1, j) - ps(i - 1, j)) / (2 * d1);
d2u = (ps(i, j + 1) - ps(i, j - 1)) / (2 * d2);
a_vx(i, j) = d1u * et1y + d2u * et2y;
a_vy(i, j) = -(d1u * et1x + d2u * et2x);
end
end
% Upper surface
j = jair;
for ii = 1:iimax;
    y(ii, jjair) = yau(ii);
end
for i = il:it
    ii = 2 * i - 1;
    jj = 2 * j - 1;
    d1x = (x(ii + 1, jj) - x(ii - 1, jj)) / d1;
    d1y = (y(ii + 1, jj) - y(ii - 1, jj)) / d1;
    d2x = (x(ii, jj + 2) - x(ii, jj)) / d2;
    d2y = (y(ii, jj + 2) - y(ii, jj)) / d2;
    jaco = d1x * d2y - d1y * d2x;
    et1x = d2y / jaco;
    et1y = -d2x / jaco;
    et2x = -d1y / jaco;
    et2y = d1x / jaco;
    d1u = (ps(i + 1, j) - ps(i - 1, j)) / (2 * d1);
    d2u = (4 * ps(i, j + 1) - 3 * ps(i, j) - ps(i, j + 2)) / (2 * d2);
    vx = d1u * et1y + d2u * et2y;
    vy = -(d1u * et1x + d2u * et2x);
    a_vx_up(i) = vx;
    a_vy_up(i) = vy;
    Vel = sqrt(vx * vx + vy * vy);
    Vru(i) = Vel / Vinf;
    Cpu(i) = 1 - Vru(i) * Vru(i);
    xup(i) = x(ii, jj);
    yup(i) = y(ii, jj);
end
% Lower surface

```

```

j = jair;
for ii = 1:iimax
    y(ii, jjair) = yal(ii);
end
for i = il:it
    ii = 2 * i - 1;
    jj = 2 * j - 1;
    d1x = (x(ii + 1, jj) - x(ii - 1, jj)) / d1;
    d1y = (y(ii + 1, jj) - y(ii - 1, jj)) / d1;
    d2x = (x(ii, jj) - x(ii, jj - 2)) / d2;
    d2y = (y(ii, jj) - y(ii, jj - 2)) / d2;
    jaco = d1x * d2y - d1y * d2x;
    et1x = d2y / jaco;
    et1y = -d2x / jaco;
    et2x = -d1y / jaco;
    et2y = d1x / jaco;
    d1u = (ps(i + 1, j) - ps(i - 1, j)) / (2 * d1);
    d2u = (-4 * ps(i, j - 1) + 3 * ps(i, j) + ps(i, j - 2)) / (2 * d2);
    vx = d1u * et1y + d2u * et2y;
    vy = -(d1u * et1x + d2u * et2x);
    a_vx_lo(i) = vx;
    a_vy_lo(i) = vy;
    Vel = sqrt(vx * vx + vy * vy);
    Vrl(i) = Vel / Vinf;
    Cpl(i) = 1 - Vrl(i) * Vrl(i);
    xlo(i) = x(ii, jj);
    ylo(i) = y(ii, jj);
end
% Plot results
figure
hold on
grid on
plot(xup, Vru, 'b', 'LineWidth', 1.5);
plot(xlo, Vrl, 'r', 'LineWidth', 1.5);
plot(xairfoil, yairfoil, 'k', 'LineWidth', 1.5);
xlabel('Chord line', 'fontsize', 14)
ylabel('Non-dimensional velocity', 'fontsize', 14)
title('velocity Coefficeint', 'fontsize', 14)
legend('Upper surface', 'Lower surface', 'Airfoil', 'Location', 'best')
figure
hold on
grid on
plot(xup, Cpu, 'b', 'LineWidth', 1.5);

```

```
plot(xlo, Cpl, 'r', 'LineWidth', 1.5);
plot(xairfoil, yairfoil, 'k', 'LineWidth', 1.5);
xlabel('Chord line', 'fontsize', 14)
ylabel('Pressure coefficient', 'fontsize', 14)
title('Pressure coefficient ', 'fontsize', 14)
legend('Upper surface', 'Lower surface', 'Airfoil', 'Location', 'best')
%% Plot Streamlines
figure
hold on
grid on
axis equal
i = 1;
ii = 2 * i - 1;
for j = 1:jmax
    jj = 2 * j - 1;
    x1(j) = x(ii, jj);
    y1(j) = y(ii, jj);
end
plot(x1, y1, 'k', 'LineWidth', 2);
i = imax;
ii = 2 * i - 1;
for j = 1:jmax
    jj = 2 * j - 1;
    x2(j) = x(ii, jj);
    y2(j) = y(ii, jj);
end
plot(x2, y2, 'k', 'LineWidth', 2);
area(x1, y1, 'FaceColor', 'k');
area(x2, y2, 'FaceColor', 'k');
j = 1;
jj = 2 * j - 1;
for i = 1:imax
    ii = 2 * i - 1;
    x3(i) = x(ii, jj);
    y3(i) = y(ii, jj);
end
plot(x3, y3, 'k', 'LineWidth', 2);
j = jmax; jj = 2 * j - 1;
for i = 1:imax
    ii = 2 * i - 1;
    x4(i) = x(ii, jj);
    y4(i) = y(ii, jj);
end
```

```
plot(x4, y4, 'k', 'LineWidth', 2);
area(x3, y3, 'FaceColor', 'k');
area(x4, y4, 'FaceColor', 'k');
plot(xairfoil, yairfoil, 'k', 'LineWidth', 2);
xlabel('X-axis', 'fontsize', 18)
ylabel('Y-axis', 'fontsize', 18)
title('Stream lines for the flow', 'fontsize', 12)
%% Plot Velocity Vector
grid on
axis equal
j = jair; jj = 2 * j - 1;
for i = il:it
    ii = 2 * i - 1;
    k = i - il + 1;
    x5(k) = x(ii, jj);
    y5(k) = yal(ii);
    plot(x5, y5);
end
plot(xairfoil, yairfoil, 'k-', 'LineWidth', 1.5);
hold on;
for i = il:it
    ii = 2 * i - 1;
    k = i - il + 1;
    x6(k) = x(ii, jj);
    y6(k) = yau(ii);
end
plot(x6, y6, 'k', 'LineWidth', 2);
hold on;
for ii = 1:iimax
    y(ii, jjair) = yau(ii);
end
for i = 1:imax
    ii = 2 * i - 1;
    for j = jair:jmax
        jj = 2 * j - 1;
        k = j - jair + 1;
        x7(i, k) = x(ii, jj);
        y7(i, k) = y(ii, jj);
        a_vx_7(i, k) = a_vx(i, j);
        a_vy_7(i, k) = a_vy(i, j);
    end
end
quiver(x7, y7, a_vx_7, a_vy_7);
```



```

hold on;
for ii = 1:iimax
    y(ii, jjair) = yal(ii);
end
for i = 1:imax
    ii = 2 * i - 1;
    for j = 1:jair
        jj = 2 * j - 1;
        x8(i, j) = x(ii, jj);
        y8(i, j) = y(ii, jj);
        p8(i, j) = ps(i, j);
    end
end
contour(x8, y8, p8, 50)
hold on;
figure
i = 1; ii = 2 * i - 1; for j = 1:jmax; jj = 2 * j - 1; x1(j) = x(ii, jj); y1(j) = y(ii, jj); end
i = imax; ii = 2 * i - 1; for j = 1:jmax; jj = 2 * j - 1; x2(j) = x(ii, jj); y2(j) = y(ii, jj); end
j = 1; jj = 2 * j - 1; for i = 1:imax; ii = 2 * i - 1; x3(i) = x(ii, jj); y3(i) = y(ii, jj); end
j = jmax; jj = 2 * j - 1; for i = 1:imax; ii = 2 * i - 1; x4(i) = x(ii, jj); y4(i) = y(ii, jj); end
plot(x1, y1, x2, y2, x3, y3, x4, y4)
hold on
xlabel('X-axis', 'fontsize', 18)
ylabel('Y-axis', 'fontsize', 18)
title('Velocity vector ', 'fontsize', 12)
j = jair; jj = 2 * j - 1;
for i = il:it; ii = 2 * i - 1; k = i - il + 1; x5(k) = x(ii, jj); y5(k) = yal(ii); end
plot(x5, y5); hold on;
for i = il:it; ii = 2 * i - 1; k = i - il + 1; x6(k) = x(ii, jj); y6(k) = yau(ii); end
plot(x6, y6); hold on;
for ii = 1:iimax; y(ii, jjair) = yau(ii); end
for i = il:it; a_vx(i, jair) = a_vx_up(i); a_vy(i, jair) = a_vy_up(i); end
for i = 1:imax
    ii = 2 * i - 1;
    for j = jair:jmax
        jj = 2 * j - 1;
        k = j - jair + 1;
        x7(i, k) = x(ii, jj);
        y7(i, k) = y(ii, jj);
        a_vx_7(i, k) = a_vx(i, j);
        a_vy_7(i, k) = a_vy(i, j);
    end
end
end

```

```

quiver(x7, y7, a_vx_7, a_vy_7)
hold on;
for ii = 1:imax; y(ii, jair) = yal(ii); end
for i = il:it; a_vx(i, jair) = a_vx_lo(i); a_vy(i, jair) = a_vy_lo(i); end
for i = 1:imax
    ii = 2 * i - 1;
    for j = 1:jair
        jj = 2 * j - 1;
        x8(i, j) = x(ii, jj);
        y8(i, j) = y(ii, jj);
        a_vx_8(i, j) = a_vx(i, j);
        a_vy_8(i, j) = a_vy(i, j);
    end
end
quiver(x8, y8, a_vx_8, a_vy_8)
hold on;
xlabel('X-axis', 'fontsize', 18)
ylabel('Y-axis', 'fontsize', 18)
title('Velocity vector ', 'fontsize', 12)
% Calculation of the lift, drag, and moment coefficients
cx = 0;
cy = 0;
cm = 0;
j = jair;
jj = jjair;
for i = il:it - 1
    ii = 2 * i - 1;
    x1 = x(ii, jj);
    y1 = yau(ii);
    x2 = x(ii + 2, jj);
    y2 = yau(ii + 2);
    cx = cx + 0.5 * (Cpu(i) + Cpu(i + 1)) * (y2 - y1) / cord;
    cy = cy - 0.5 * (Cpu(i) + Cpu(i + 1)) * (x2 - x1) / cord;
    cm = cm + 0.5 * (Cpu(i) + Cpu(i + 1)) * ((x2 - x1) * (0.25 - x1/cord));
    x1 = x(ii, jj);
    y1 = yal(ii);
    x2 = x(ii + 2, jj);
    y2 = yal(ii + 2);
    cx = cx + 0.5 * (Cpl(i) + Cpl(i + 1)) * (y2 - y1) / cord;
    cy = cy + 0.5 * (Cpl(i) + Cpl(i + 1)) * (x2 - x1) / cord;
    cm = cm + 0.5 * (Cpl(i) + Cpl(i + 1)) * ((x2 - x1) * (0.25 - x1/cord));
end
cl = cy * cosa - cx * sina;

```

```
cd = cy * sina + cx * cosa;
cm0_25c = cm;
disp(['Lift coefficient (cl): ', num2str(cl)])
disp(['Drag coefficient (cd): ', num2str(cd)])
disp(['Moment coefficient (cm0.25c): ', num2str(cm0_25c)])
end
```

#### [7] tri\_sol.m

```
function e = tri_sol(a, b, c, d, M)
% Forward elimination
for i = 2:M
    t = b(i) / d(i - 1);
    d(i) = d(i) - t * a(i - 1);
    c(i) = c(i) - t * c(i - 1);
end
% Back substitution
e = zeros(M, 1);
e(M) = c(M) / d(M);
for k = 2:M
    i = M - k + 1;
    e(i) = (c(i) - a(i) * e(i + 1)) / d(i);
end
end
```

#### [8] Laplace\_main.m

```
function laplace_main
clear all; close all; clc;
global x y imax jmax jair il it cord yal yau ps psp dx dy r d1 d2 omega Vinf cosa sina
% Data
Vinf = 100; alfad = 5; cord = 1; nitd = 0;
il = 31; it = 71; imax = 101; jair = 26; jmax = 51;
omega = 1; per = .000001; nmax = 20;
% Derived quantities
alfa = alfad * pi / 180;
cosa = cos(alfa); sina = sin(alfa);
uxinf = Vinf * cosa; uyinf = Vinf * sina;
iimax = 2 * imax - 1; jjmax = 2 * jmax - 1; jjair = 2 * jair - 1;
lx = 3 * cord; ly = cord; M = imax;
nx = imax - 1; ny = jmax - 1; d1 = 1 / (it - il); d2 = 1 / ny;
dx = cord / (it - il); dy = ly / ny;
r = dx / dy; t1 = omega / (2 * (1 + r * r));
t2 = t1 * r * r;
geom
xxx = 1; % "if you want the SOR by Line method"
% Boundary values & Initialization
```

```

ps = zeros(imax, jmax);
psp = zeros(imax, jmax);
i = 1;
for j = 1:jmax - 1
    ii = 2 * i - 1; jj = 2 * j - 1;
    ps(i, j + 1) = ps(i, j) + uxinf * (y(ii, jj + 2) - y(ii, jj));
end
j = 1;
for i = 1:imax - 1
    ii = 2 * i - 1; jj = 2 * j - 1;
    ps(i + 1, j) = ps(i, j) - uyinf * (x(ii + 2, jj) - x(ii, jj));
end
i = imax;
for j = 1:jmax - 1
    ii = 2 * i - 1; jj = 2 * j - 1;
    ps(i, j + 1) = ps(i, j) + uxinf * (y(ii, jj + 2) - y(ii, jj));
end
j = jmax;
for i = 1:imax - 1
    ii = 2 * i - 1; jj = 2 * j - 1;
    ps(i + 1, j) = ps(i, j) - uyinf * (x(ii + 2, jj) - x(ii, jj));
end
for i = 2:imax - 1
    for j = 2:jmax - 1
        ps(i, j) = ps(i, 1) + (ps(i, jmax) - ps(i, 1)) * (j - 1) / (jmax - 1);
    end
end
psp = ps;
% Iterative process
a_n = zeros(nmax, 1);
a_lmder = zeros(nmax, 1);
a_rms = zeros(nmax, 1); % Array to store RMS errors
for n = 1:nmax
    % Calculation of new values of ps(i,j) = psp(i,j)
    if xxx == 1; L_SOR; end
    % Errors calculation
    mder = 0;
    sum_sq_error = 0; % Initialize sum of squared errors
    count = 0; % Initialize counter for internal points
    for i = 2:nx
        for j = 2:ny
            der = abs(psp(i, j) - ps(i, j));
            sum_sq_error = sum_sq_error + der^2; % Accumulate squared error
        end
    end
end

```

```
        count = count + 1; % Increment counter
        if der > mder; mder = der; end
        ps(i, j) = psp(i, j);
    end
end
% Calculate RMS error
rms_error = sqrt(sum_sq_error / count);
a_rms(n) = rms_error;
if mder > 0; lmder = log10(mder); end
% Updating the value of psi on the airfoil
psold = psp(it, jair);
psnew = psp(it + 1, jair);
erps = abs(psnew - psold);
j = jair;
for i = il:it
    ps(i, j) = psnew;
    psp(i, j) = psnew;
end
a_n(n) = n; a_lmder(n) = lmder;
end
% Plot convergence log with RMS error
figure
plot(a_n, a_rms)
grid on
xlabel('Iteration number', 'fontsize', 18)
ylabel('RMS Error', 'fontsize', 18)
title(['Convergence history using LSOR '], 'fontsize', 12)
results
toc
% Load the airfoil data and call the plotting function
airfoil_data = load('airfoil_data.txt');
x = airfoil_data(:, 1);
y = airfoil_data(:, 2);
cp = airfoil_data(:, 2);
plot_iso_lines(x, y, cp);
end
```

## VIII. References

[1] <https://pressbooks.online.ucf.edu/aerodynamicstext/chapter/chapter-5-theory-of-airfoil-aerodynamics/>

- [2]<https://aviation.stackexchange.com/questions/5230/how-to-plot-the-pressure-distribution-over-an-airfoil>
- [3]<http://airfoiltools.com/index>